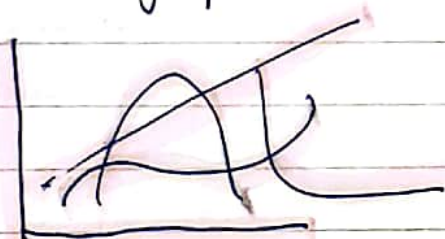


9-dars: Optimization

Optimization

- Complex datasetlarda (asosan) bitta algoritm-larni yechishda yaroqli emas.

↳ Borgan sari dataset / complexlashib, stabil bo'lmayapti.



⇒ Yani xur - ril strukturalik bo'lishi mumkin. Albatta

Diyimatlari xur - xilligi u - n. Shuning uchun biz "Optimizationni" o'rganish va "yollashga" tabiiy ehtiyoj tug'uladi.

- Har bir algoritmni o'zini kuchli va kechsiq tomonlari bor.

- Ensemble algoritmilar = $A_1 + A_2 + A_3$

↳ Ensemble algoritmilar "modelni" optimallashtirish uchun eng foydali yollardan biri hisoblanadi.

↳ Accuracy

↳ Stability

↳ Generalization

• Optimization = model uchun eng xos bo'lgan parameterlarni topib beradi.

• Maqsad: **Errorni** kamaytirish / **accuracy**ni oshirish va ~~at~~ qalbatta **generalization**ni oshiradi.

→ ① Model parameter optimization (learned automatically)

② Hyperparameter optimization (set manually)

Datasetda o'zgaradi "training" jarayonida

→ Biz qolida "manually set" qilamiz.

Why is this so important?

① Avoids underfitting and overfitting.

② Improves model accuracy and stability.

③ Reduces training time by using efficient

~~④ Essential~~ settings.

④ Essential for achieving state-of-the-art performance.

Ensemble models

- Kuchli model hosil qilish uchun bir nechta algoritmlarni birlashtirish.

Ensemble Learning

- Asosiy foydasi:

① Accurate



② Stable



③ Robust # Generalization
→ qat'iylik, chidamlilik, bardoshlik



Model ma'lumotlar buzilganda, muhit o'zgarib qando, yoki ketilmagan kiritilgan ma'lumotlar tizimda o'z ishliligini qanchalik ushlab qola olishini ko'rsatadi (Noise & Outliers, Distribution Shift / Data Shift, and Adversarial Robustness)

- Ensemble algoritmlar "ensemble" methodlar yordamida yaratiladi.

① Bagging (Bootstrapping)



② RF, Bagging Reg / Class

- Boosting (Ketma-ket undan oldingisini natijalarini to'g'irlab ketadi)



① AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost

◦ Voting

① Simple rule (sodda modellarni) orqali prediction/ni birlashtiradi.

② Soft va Hard Voting

③ Ensemble algorithms

④ Prediction/ni birlashtiradi (base learners) va yakuniy prediction chiqaradi.
 Bir nechta algoritm/ni birlashtiradi dedik
 Bu fan tilida "base learners" d-di.

⑤ Agar turli xil algoritm/da error/ turli xil chiqib ketayotgan bolsa ishlatilmasligi ilinadi.

How does Voting work?

◦ Hujan bir xil datasetda turli xil base algorithm (base learners) "train" qilinadi.

◦ Logistic Regression, DT, SVC dan tashkil topish m-n.

◦ Yakuniy predictionda:

↳ ① Combine qiladi (voting rule) --- Classification

↳ ② Classificationda "Voting" 2 turga bo'linadi:
 "Hard Voting" va "Soft Voting"

Algos	Prediction
LR	Yes
DT	No
SVC	No

→ Result: "No" → Bu Hard Voting

Algos	P(YES)	P(No)
LR	0.7	0.3
DTC	0.6	0.4
SVC	0.4	0.6

⇒ Soft Voting (Probability)

① P(YES) ni o'zta arifmetigi topiladi.

$$P(\text{YES})_{\text{mean}} = \frac{0.7 + 0.6 + 0.4}{3} \approx 0.56$$

② P(No) ni o'zta arifmetigi :

$$P(\text{No})_{\text{mean}} = \frac{0.3 + 0.4 + 0.6}{3} \approx 0.4$$

③ $P(\text{YES})_{0.56} > P(\text{No})_{0.4}$ javoblar YES tarafga oladi.

! Agar dataset balanslashgan bo'lsa "Soft Voting" ishlatgan ma'qul.

! Agar dataset balanslashmagan bo'lsa "Hard Voting" ishlatgan ma'qul.

Voting (Classifier) ishlatayotganimizda "Ham Hard Voting ham Soft Voting" algoritmini ishlatish boladi.

• Regression --- "Mean"

VOTING CLASSIFIER

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.ensemble import VotingClassifier
```

```
X = df.drop("red_cards", axis=1)
y = df["red_cards"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

```
lr = LogisticRegression(max_iter=1000)
dt = DecisionTreeClassifier()
svm = SVC(probability=True) # Soft Voting u-n
```

Hard Voting

```
hard_voting = VotingClassifier(
    # Base learners ← estimators = [ ("lr", lr), ("dt", dt), ("svm", svm) ],
    voting = "hard")
```

```
hard_voting.fit(X_train, y_train)
```

```
y_pred_hard = hard_voting.predict(X_test)
```

```
print("Hard Voting Accuracy:", accuracy_score(y_test,
    y_pred_hard))
```

Soft Voting

```
soft_voting = VotingClassifier (
```

```
#Base learners < estimators = ["lr", lr], ["dt", dt], ["svm", svm],  
    voting = "soft"
```

```
)  
soft_voting.fit(X_train, y_train)
```

```
y_pred_soft = soft_voting.predict(X_test)
```

```
print("Soft Voting Accuracy:", accuracy_score(  
    y_test, y_pred_soft))
```

```
print("\n Soft Voting: Predicted Probabilities (first 5  
    sample):")
```

```
print(soft_voting.predict_proba(X_test[:5]))
```

Regression bôgicha o'layapmiz.

```
from sklearn.linear_model import LinearRegression, Ridge  
from sklearn.tree import DecisionTreeRegressor  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import mean_squared_error,  
    r2_score
```

```
X = df.drop("X6", axis=1)
```

```
y = df["X6"]
```

```
X_train, X_test, y_train, y_test = train_test_split
```

```
(X, y, test_size = 0.3, random_state = 42)
```

```
lr = LinearRegression()
```

```
ridge = Ridge()
```

```
dt = DecisionTreeRegressor()
```

```
# Voting Regressor
```

```
voting_reg = VotingRegressor()
```

```
estimators = [("lr", lr), ("ridge", ridge),  
              ("dt", dt)]
```

```
voting_reg.fit(X_train, y_train)
```

```
y_pred = voting_reg.predict(X_test)
```

```
print("Voting Regressor MSE:", mean_squared_error(y_test, y_pred))
```

```
print("Voting Regressor R2:", r2_score(y_test, y_pred))
```

```
# Show predictions for the first 5 samples
```

```
print("\n Voting Regressor Predictions (first 5  
samples):")
```

```
print(y_pred[:5])
```